

Magnafic AI

Stakeholder Technology Overview

A business-friendly explanation of what we used, why we used it, and how the platform supports AI-assisted consulting, sales, distribution, research, knowledge retrieval, and reporting.

Purpose: help non-technical stakeholders understand the platform stack and the role each technology plays.

1. Executive Summary

Magnafic AI has been built as a modern AI platform, not as a simple chatbot. It combines a web application, secure backend APIs, project lifecycle management, document knowledge retrieval, AI model routing, background processing, and an agent runtime. This lets the platform support structured business workflows such as onboarding, company discovery, AI chat, research planning, report generation, and future sales/distribution automation.

- The frontend is a Next.js web portal where users manage projects, knowledge, research, reports, and AI workforce agents.
- The backend is a NestJS API that controls authentication, business logic, data access, AI execution, and admin visibility.
- PostgreSQL with pgvector stores business records and searchable AI knowledge embeddings.
- Redis and BullMQ support background jobs such as discovery, document processing, and agent execution.
- LiteLLM is used as the model gateway so the platform can route AI calls through configurable model providers.
- LangGraph concepts support controlled agent workflows with planning, specialist execution, and verification.
- Sanity can manage workforce agent profiles without requiring code deployment for every agent instruction change.

2. What We Have Used

Layer	Technology Used	What It Does	Stakeholder Value
Web experience	Next.js, React, TypeScript	Builds the user-facing platform: login, dashboard, projects, onboarding, knowledge, chat, agents, reports, and admin screens.	Gives teams a structured web portal instead of scattered AI tools.
Backend API	NestJS, TypeScript	Provides secure business APIs and coordinates project, knowledge, research, reporting, and agent workflows.	Keeps the platform maintainable and ready for enterprise workflows.
Database	PostgreSQL, Prisma, pgvector	Stores projects, users, profiles, documents, conversations, agent runs, audit logs, reports, and vector embeddings.	Creates a reliable system of record for business and AI activity.
Authentication	Firebase Auth, JWT, RBAC	Verifies user identity and protects routes by authenticated role and context.	Improves trust, access control, and security readiness.
AI model access	LiteLLM gateway	Routes requests to configured AI models without tying the system to one provider.	Allows flexibility in model selection, cost control, and future vendor changes.
Agent runtime	LangGraph-style runtime, specialists, verifier	Coordinates AI work through supervised steps, reusable specialists, and final answer verification.	Supports business agents that are more controlled than a generic chatbot.
Knowledge retrieval	RAG, embeddings, vector search	Lets AI retrieve relevant project documents and company knowledge before answering.	Improves answer relevance and reduces unsupported responses.
Background jobs	Redis, BullMQ	Runs longer tasks such as discovery and document processing outside the user request.	Makes the platform more responsive and scalable.
Content / agent config	Sanity Studio	Stores optional workforce agent profiles, instructions, capabilities, and output rules.	Lets business/admin teams update agent behavior more safely.
Quality checks	Vitest, Playwright, ESLint, TypeScript	Checks code behavior, browser flows, formatting, and type safety.	Reduces defects before release.

3. Platform Flow in Simple Terms

The platform works as a connected business workflow. A user signs in, creates a project, adds company information, lets the system discover and organize context, uploads knowledge documents, and then uses AI agents for research, chat, and reporting.

Step	What Happens	Technology Involved	Business Outcome
1	User signs in and receives secure platform access.	Firebase Auth, JWT, RBAC	Only authorized users can access project data.
2	User creates a project and completes onboarding.	Next.js, NestJS, PostgreSQL	The platform captures company and project context.
3	Discovery jobs process website/company information.	BullMQ, Redis, discovery services, LiteLLM	Company profile and research sources are prepared.
4	Documents are uploaded and converted into searchable knowledge.	Storage, text extraction, embeddings, pgvector	AI can use uploaded business material when answering.
5	AI agents answer questions or generate research/report outputs.	LangGraph-style runtime, RAG, LiteLLM	Users get contextual, structured business outputs.
6	Admin screens monitor health, users, audit logs, tools, and agent runs.	Admin APIs, audit logs, database records	Stakeholders gain visibility and operational control.

4. Why These Technologies Were Chosen

Decision	Reason
Use TypeScript across web and API	One language across frontend and backend improves developer productivity, type safety, and long-term maintainability.
Use NestJS for backend	NestJS supports modular architecture, dependency injection, guards, validation, and enterprise-style API organization.
Use PostgreSQL with pgvector	PostgreSQL is reliable for business data, while pgvector allows AI knowledge search in the same database foundation.
Use LiteLLM	The platform avoids hard-coding to a single AI provider and can route models by workload, cost, and quality.
Use RAG	Business answers need grounding in project documents and company context, not only model memory.
Use Redis/BullMQ	AI and document workflows can take time, so background queues keep the user experience responsive.
Use Sanity for workforce agents	Agent profiles and instructions can evolve without deep code changes for every business role.
Use Playwright and Vitest	Automated tests help protect user workflows and backend logic as the product grows.

5. What Is Not Required Right Now

Several agent and automation technologies are useful to know, but they are not required in the current Magnafic AI implementation. They may become relevant later depending on integration and automation goals.

Technology	Current Need	Reason
CrewAI	Not required now	The platform already has its own business-agent runtime and specialist architecture.
OpenAI Agents SDK	Not required now	AI execution is currently routed through LiteLLM and the platform runtime.
Microsoft Agent Framework / AutoGen / Semantic Kernel	Not required now	Useful only if a Microsoft-first agent stack is selected later.
LlamaIndex	Optional future learning	Current RAG pipeline is custom using PostgreSQL, pgvector, and internal services.
n8n	Future integration option	Useful for connecting external SaaS workflows, but not core to the current build.
UiPath / RPA	Future automation option	Useful for legacy screen automation where APIs are unavailable.
FastAPI	Not required	The backend is NestJS, not Python/FastAPI.
OCR / IDP	Future enhancement	Useful for scanned invoices and claims; current document flow focuses on text extraction and embeddings.

6. Stakeholder View of Business Benefits

- Faster project setup: onboarding captures company context in a reusable structure.
- Better AI answers: RAG lets agents consult project-specific documents and profiles.
- Reusable business agents: sales, marketing, finance, legal, production, and future agents can share one controlled runtime.
- Operational visibility: agent runs, tool executions, audit logs, and admin health checks make activity reviewable.
- Future integration readiness: the architecture can connect to ERP, CRM, SFA, DMS, distributor portals, and workflow tools through APIs or automation layers.
- Vendor flexibility: LiteLLM keeps the AI model layer configurable instead of locking the business into one model provider.

7. Suggested Learning and Roadmap

Priority	Area	Why It Matters
1	TypeScript, APIs, SQL	Core skills for understanding and extending the existing product.
2	NestJS, Prisma, PostgreSQL	Main backend architecture and data layer.
3	RAG, embeddings, pgvector	Foundation for project knowledge and contextual AI responses.
4	LangGraph concepts	Useful for understanding agent planning, routing, state, and verification.
5	LiteLLM routing	Important for model selection, fallback, cost, and provider flexibility.
6	Redis/BullMQ jobs	Important for scalable discovery, document processing, and long-running AI tasks.
7	Playwright, tests, observability	Required to make the platform reliable and measurable.
8	n8n, UiPath, browser automation	Consider when integrating with external systems that lack clean APIs.

8. Final Takeaway

The current Magnafic AI stack is appropriate for a serious AI business platform. It uses a secure web application, enterprise-style backend, structured database, vector search, background jobs, configurable model gateway, and controlled agent runtime. Additional tools such as n8n, UiPath, CrewAI, LlamaIndex, or Microsoft agent frameworks are not mandatory today; they should be considered only when a specific business integration or automation requirement appears.